

Chapter 37

Supervised Learning Algorithms

Explain how each major supervised learning algorithm makes a prediction, and calculate one small step of each by hand · fit and read linear regression, ridge, lasso, and elastic net · fit and read logistic regression, including odds ratios

Approved chapter · v1

Contents

Chapter 37. Supervised Learning Algorithms

Why this matters

In plain English

37.0 The data for this chapter

37.1 Linear regression

37.2 Regularization: ridge, lasso, and elastic net

37.3 Logistic regression

37.4 k-nearest neighbors

37.5 Naive Bayes

37.6 Decision trees

37.7 Random forests

37.8 Gradient boosting

37.9 Support vector machines

37.10 Bias, variance, and learning curves

37.11 Hyperparameter tuning

37.12 Project result: lead scoring, baseline vs boosting

Which algorithm when

Common mistakes and how to spot them

In the real world: "Should we buy the AutoML tool?"

Tools

The project: lead scoring, baseline vs boosting

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 37. Supervised Learning Algorithms

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: explain how each major supervised learning algorithm makes a prediction, and calculate one small step of each by hand · fit and read linear regression, ridge, lasso, and elastic net · fit and read logistic regression, including odds ratios · use k-nearest neighbors and Naive Bayes, and know their weak spots · build a decision tree split by hand with Gini and entropy, and see a tree overfit · explain why random forests and gradient boosting work, and use scikit-learn, XGBoost, LightGBM, and CatBoost · know when support vector machines are worth it · diagnose bias and variance with learning curves · tune hyperparameters with random search and Optuna without fooling yourself · pick an algorithm for a problem, and justify the choice.

Before you start: Chapter 35 (loss, gradients, likelihood, entropy) and Chapter 36 (splits, cross-validation, leakage, pipelines, baselines). This chapter reuses Chapter 36's pipeline and lead-scoring data.

Time needed: 14–18 hours over two to three weeks. It's the longest chapter in Part IV; take it one algorithm at a time.

Tools: Python 3 with scikit-learn, plus three free gradient-boosting libraries (XGBoost, LightGBM, CatBoost) and Optuna for tuning. Everything runs on a laptop CPU; the slowest block takes about a minute.

Practice data: two Riverstone datasets. **Customer accounts** (new in this chapter, built by `companion/generate_riverstone_accounts.py`): 5,000 B2B accounts described as of 31 December 2024, with what happened in 2025. **Leads** (from Chapter 36), loaded through `companion/ch37/lead_data.py`. Every number in this chapter was calculated, and every output shown is real.

Why this matters

There are hundreds of machine learning algorithms, and a handful of them do almost all the work in business data science. Interviewers expect you to know those few well: not only the function name, but how each one decides, what goes wrong, and which settings matter.

The practical questions come up every week:

- "Should we use XGBoost?" Maybe. On one of this chapter's datasets it clearly beats logistic regression; on the other it doesn't.

- "The model is 99% accurate on training data but poor on new data." That's variance, and you'll learn to see it on a chart.
- "Which features drive churn?" Some algorithms tell you directly; others need extra tools (Chapter 39).
- "How long should we spend tuning?" Less than most people think, and never on the test set.

This chapter teaches nine algorithm families, one at a time, on real Riverstone problems. For each one, you'll see how it works, calculate a small piece by hand, run it, and read the result. At the end, you'll compare them fairly and learn to choose.

In plain English

Think of different people guessing which customers will stop ordering.

- The **accountant** gives every warning sign a weight and adds them up: "late payments +3, no recent order +5, many product lines -2". That's **linear and logistic regression**.
- The **accountant with a strict boss** has to justify every weight, so they keep them small and drop the ones that barely matter. That's **regularization**.
- The **old-timer** says, "This customer reminds me of twenty others I've known; fifteen of them left." That's **k-nearest neighbors**.
- The **statistician** asks, "Among customers who left, how common was this sign? Among those who stayed?" and multiplies the evidence. That's **Naive Bayes**.
- The **sales manager** asks a series of yes/no questions: "No order in 90 days? Pays late? Small retailer?" That's a **decision tree**.
- A **committee** of 300 managers, each seeing slightly different customers and questions, votes. That's a **random forest**.
- A **relay team**, where each new member studies the mistakes of everyone before them and fixes a little of what they got wrong. That's **gradient boosting**.
- The **boundary drawer** looks for the widest possible gap between customers who left and customers who stayed. That's a **support vector machine**.

None of them is always best. The skill is knowing which to ask, and how to check whether they're right.

37.0 The data for this chapter

Riverstone customer accounts

Chapter 36 scored new leads. This chapter adds the other half of the customer life cycle: **accounts**, businesses that already buy from Riverstone. Each row describes one account as of 31 December 2024, the **prediction moment**, using only what was known then: segment, city tier, size, sales rep, tenure,

2024 orders and revenue, discounts, late payments, complaints, how many product categories it buys, and days since its last order. Two targets describe 2025:

- **churned_2025** : 1 if the account placed no order in 2025. A classification target.
- **revenue_2025** : the account's 2025 revenue. A regression target.

Because every account is described at the same moment and predicted over the same year, a stratified random split is appropriate here (Chapter 36's time split mattered because leads arrive over time, and the model scores future leads).

```
import warnings

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import log_loss, mean_absolute_error, r2_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

warnings.filterwarnings("ignore", category=UserWarning)

accounts = pd.read_csv("../accounts/accounts.csv")
accounts["rep_id"] = accounts["rep_id"].astype(str)
print(accounts.shape)
print(
    f"churned in 2025: {accounts['churned_2025'].sum():,} of {len(accounts):,} "
    f"({accounts['churned_2025'].mean():.1%})"
)

CATS = ["segment", "city_tier", "company_size", "rep_id"]
NUMS = [
    "tenure_months",
    "orders_2024",
    "units_2024",
    "revenue_2024",
    "avg_discount_pct",
    "late_payment_days",
    "complaints_2024",
    "categories_bought",
    "days_since_last_order",
    "website_logins_2024",
    "catalog_downloads_2024",
]

rest, test_acc = train_test_split(
    accounts, test_size=0.2, random_state=37, stratify=accounts["churned_2025"]
)
train_acc, valid_acc = train_test_split(
    rest, test_size=0.25, random_state=37, stratify=rest["churned_2025"]
)
for name, part in [("train", train_acc), ("valid", valid_acc), ("test", test_acc)]:
    print(
        f"{name:<6} {len(part):>5,} accounts    churn {part['churned_2025'].mean():.1%}"
    )
```

```
(5000, 20)
churned in 2025: 484 of 5,000 (9.7%)
train 3,000 accounts churn 9.7%
valid 1,000 accounts churn 9.7%
test 1,000 accounts churn 9.7%
```

Reading it. 484 of 5,000 accounts, 9.7%, stopped ordering in 2025. The stratified split keeps that rate identical in the training (3,000), validation (1,000), and test (1,000) sets. The test set stays locked until section 37.11.

A fact you can't normally know. Because this dataset comes from a generator, we know the true churn probability of every account. A perfect model that knew those probabilities would score an AUC of **0.858** on these accounts. No real model can do better, because churn is partly chance. It's a useful ruler: in real work you never see this number, so you can't tell how close to the ceiling you are.

37.1 Linear regression

How it works

Linear regression predicts a number as a weighted sum of features plus an intercept:

$$\text{prediction} = b + w_1 \times \text{feature}_1 + w_2 \times \text{feature}_2 + \dots + w_n \times \text{feature}_n$$

It chooses the weights that minimize the mean squared error (Chapter 35). Unlike gradient descent, for linear regression there's an exact formula, the **least-squares solution**, so scikit-learn solves it directly in one step.

Predicting 2025 revenue. Money amounts are skewed (Chapter 35), so model the **log** of revenue. In a log-log model, a slope reads as a percentage relationship. Start with one feature: last year's revenue. Churned accounts have no 2025 revenue, so the regression uses the 4,516 accounts that stayed; predicting *whether* an account stays is the classification problem later in the chapter.

```

stayed = accounts[accounts["churned_2025"] == 0].copy()
for col in ["revenue_2024", "units_2024", "orders_2024", "revenue_2025"]:
    stayed["log_" + col.replace("_2025", "_next")] = np.log(stayed[col])
stayed = stayed.rename(columns={"log_revenue_next": "log_revenue_2025"})
reg_train, reg_test = train_test_split(stayed, test_size=0.25, random_state=37)
print(len(reg_train), "training accounts,", len(reg_test), "test accounts")

from sklearn.linear_model import LinearRegression

simple = LinearRegression().fit(
    reg_train[["log_revenue_2024"]], reg_train["log_revenue_2025"]
)
print(f"intercept {simple.intercept_:.3f}    slope {simple.coef_[0]:.3f}")
pred = simple.predict(reg_test[["log_revenue_2024"]])
print(f"R² on test: {r2_score(reg_test['log_revenue_2025'], pred):.4f}")
example = reg_test.iloc[0]
print(
    f"one account: 2024 ₹{example['revenue_2024']:.0f}"
    f" -> predicted 2025 ₹{np.exp(pred[0]):.0f}"
    f" (actual ₹{example['revenue_2025']:.0f})"
)

```

```

3387 training accounts, 1129 test accounts
intercept -0.140    slope 1.013
R² on test: 0.9546
one account: 2024 ₹65,500 -> predicted 2025 ₹65,555 (actual ₹71,100)

```

Reading it.

- **Slope 1.013:** an account with 1% more revenue in 2024 is predicted to have about 1.013% more in 2025. Revenue carries forward almost one for one.
- **R² 0.9546:** R² (the **coefficient of determination**) is the share of the variation in the target that the model explains, from 0 (no better than predicting the average) to 1 (perfect). Last year's revenue alone explains 95% of the variation in log revenue this year, which is typical: the biggest predictor of next year is this year.
- **The example account:** 2024 revenue of ₹65,500, predicted ₹65,555 for 2025, actual ₹71,100. Check the arithmetic: $\ln(65,500) = 11.0898$; $-0.140 + 1.013 \times 11.0898 = 11.0940$ (with the unrounded coefficients, 11.0906); $e^{11.0907} = ₹65,555$. ✓

Many features

```
REG_NUMS = [
    "log_revenue_2024",
    "log_units_2024",
    "log_orders_2024",
    "tenure_months",
    "avg_discount_pct",
    "late_payment_days",
    "complaints_2024",
    "categories_bought",
    "days_since_last_order",
    "website_logins_2024",
    "catalog_downloads_2024",
]

def reg_pipeline(model):
    prepare = ColumnTransformer(
        [
            ("cat", OneHotEncoder(handle_unknown="ignore", drop="first"), CATS),
            (
                "num",
                Pipeline(
                    [
                        ("fill", SimpleImputer(strategy="median")),
                        ("scale", StandardScaler()),
                    ]
                ),
                REG_NUMS,
            ),
        ]
    )
    return Pipeline([("prepare", prepare), ("model", model)])

def reg_report(name, pipe):
    pipe.fit(reg_train[CATS + REG_NUMS], reg_train["log_revenue_2025"])
    pred = pipe.predict(reg_test[CATS + REG_NUMS])
    mae = mean_absolute_error(reg_test["revenue_2025"], np.exp(pred))
    print(
        f"{name:<18} R² {r2_score(reg_test['log_revenue_2025'], pred):.4f}    "
        f"MAE ₹{mae:,.0f}"
    )
    return pipe

ols = reg_report("linear regression", reg_pipeline(LinearRegression()))
names = ols.named_steps["prepare"].get_feature_names_out()
coefs = pd.Series(ols.named_steps["model"].coef_, index=names)
print(coefs.round(3).sort_values().to_string())
```

```
linear regression R² 0.9593 MAE ₹87,471
cat__segment_Retail -0.097
cat__segment_Wholesale -0.096
num__late_payment_days -0.048
cat__city_tier_Tier 2 -0.041
cat__city_tier_Tier 3 -0.036
num__complaints_2024 -0.033
num__website_logins_2024 -0.011
num__days_since_last_order -0.006
num__log_units_2024 -0.001
num__log_orders_2024 -0.001
cat__company_size_Small -0.001
num__tenure_months 0.000
cat__company_size_Medium 0.004
cat__rep_id_4 0.004
cat__rep_id_5 0.007
num__catalog_downloads_2024 0.008
num__avg_discount_pct 0.012
num__categories_bought 0.025
cat__rep_id_9 0.031
num__log_revenue_2024 1.099
```

How it works:

- `OneHotEncoder(drop="first")` drops one category per column, so `segment` becomes two columns (Retail, Wholesale) compared against the dropped one, **Hospitality**. Linear regression needs this: with all three columns plus an intercept, the columns always add up to 1, and the weights can't be pinned down (the **dummy variable trap**). Regularized models and trees don't need it.
- Numbers are standardized, so each numeric coefficient is "the change in log revenue for one standard deviation more of this feature". That makes coefficients comparable with each other.
- **MAE** (mean absolute error) converts predictions back to rupees and averages the size of the misses: ₹87,471. Chapter 39 covers regression metrics in full.

Reading the coefficients.

- `log_revenue_2024` dominates (1.099), as expected.
- `segment_Retail` -0.097: holding everything else equal, a retail account's 2025 revenue is predicted about 9% lower than a hospitality account's ($e^{-0.097} = 0.908$). The generator planted faster growth for hospitality, especially in metro cities.
- `late_payment_days` -0.048 and `complaints_2024` -0.033: accounts that pay late or complain grow less.
- `log_units_2024` and `log_orders_2024` are almost exactly 0. They carry nearly the same information as revenue, so once revenue is in the model, they add nothing. When features are strongly correlated (**multicollinearity**), their individual coefficients become unstable and hard to interpret, even when predictions are fine.
- `rep_id_9` +0.031: accounts handled by the inside sales desk are predicted to grow 3% faster. The generator planted **no** such effect on growth. Exercise 13 shows its confidence interval, and a likely reason it appears: this regression only includes accounts that *stayed*. A coefficient needs an interval and a second look before anyone acts on it; `statsmodels` gives the interval, and Chapter 22 explains how to read it.

Watch out: a coefficient is not a cause. "Late payments reduce growth by 5% per standard deviation" is a statement about this model, not about the world. Maybe late payers are struggling businesses that would shrink anyway. Chapter 31 covers methods for estimating causes from observational data.

Linear regression's assumptions, and what breaks when they fail:

Assumption	If it fails	Check
The relationship is linear (after any transforms)	Systematic errors in some ranges	Plot residuals against predictions
Errors are independent	Overconfident intervals	Think about repeated customers and time
Errors have constant spread	Intervals too narrow for big accounts	Residual plot fans out; use a log target
Features aren't nearly duplicates	Unstable, uninterpretable coefficients	Correlations between features; regularization
No extreme outliers dominate	One account drags the line	Look at the largest residuals

For prediction alone, the first and last matter most. For interpreting coefficients, all of them matter.

37.2 Regularization: ridge, lasso, and elastic net

The idea

With many features, especially correlated ones or few rows, least squares can give large, unstable weights that fit noise. **Regularization** adds a penalty for large weights to the loss, trading a little training fit for more stable predictions:

Ridge: $\text{loss} = \text{mean squared error} + \alpha \times (\text{sum of squared weights})$

Lasso: $\text{loss} = \text{mean squared error} + \alpha \times (\text{sum of absolute weights})$

Elastic net: a mix of both penalties, set by `l1_ratio`

α (alpha) controls the strength. At $\alpha = 0$ you get ordinary least squares; as α grows, weights shrink toward zero.

The two penalties behave differently. **Ridge** shrinks all weights a little and keeps every feature; with correlated features, it spreads the weight among them. **Lasso** pushes some weights to **exactly zero**, removing those features, so it doubles as **feature selection**. Elastic net sits between them, and handles groups of correlated features more gracefully than lasso, which tends to pick one at random.

Because the penalty treats every weight equally, **features must be standardized first**; otherwise a feature measured in rupees gets a tiny weight and escapes the penalty.

The `...CV` versions choose α by cross-validation inside the training data:

```
from sklearn.linear_model import ElasticNetCV, LassoCV, RidgeCV

ridge = reg_report(
    "ridge (RidgeCV)", reg_pipeline(RidgeCV(alphas=np.logspace(-3, 3, 25)))
)
lasso = reg_report("lasso (LassoCV)", reg_pipeline(LassoCV(cv=5, random_state=37)))
enet = reg_report(
    "elastic net",
    reg_pipeline(ElasticNetCV(l1_ratio=[0.2, 0.5, 0.8], cv=5, random_state=37)),
)
print(
    "ridge alpha:",
    ridge.named_steps["model"].alpha_,
    "lasso alpha:",
    round(lasso.named_steps["model"].alpha_, 5),
    "elastic net l1_ratio:",
    enet.named_steps["model"].l1_ratio_,
)
lasso_coefs = pd.Series(lasso.named_steps["model"].coef_, index=names)
print("lasso set to exactly zero:", list(lasso_coefs[lasso_coefs == 0].index))
```

```
ridge (RidgeCV)    R² 0.9593    MAE ₹87,466
lasso (LassoCV)    R² 0.9594    MAE ₹87,463
elastic net        R² 0.9594    MAE ₹87,438
ridge alpha: 0.1   lasso alpha: 0.0011   elastic net l1_ratio: 0.8
lasso set to exactly zero: ['cat__company_size_Medium', 'cat__rep_id_4', 'cat__rep_id_5',
                             'num__log_units_2024', 'num__log_orders_2024', 'num__tenure_months']
```

Reading it. All four models score almost the same (R^2 0.9593–0.9594). That's the honest result for this dataset: with 3,387 training rows and 20 features, ordinary least squares isn't overfitting, so there's little for regularization to fix. What lasso does do is **simplify**: it removed six features, including `log_units_2024` and `log_orders_2024` (duplicates of revenue) and `tenure_months`, without losing any accuracy.

Increase α by hand to see the trade-off:

```
from sklearn.linear_model import Lasso

for alpha in [0.001, 0.01, 0.05, 0.2]:
    m = reg_pipeline(Lasso(alpha=alpha)).fit(
        reg_train[CATS + REG_NUMS], reg_train["log_revenue_2025"]
    )
    kept = (m.named_steps["model"].coef_ != 0).sum()
    r2 = r2_score(reg_test["log_revenue_2025"], m.predict(reg_test[CATS + REG_NUMS]))
    print(
        f"alpha {alpha:<5}  features kept {kept:>2} of {len(names)}  test R² {r2:.4f}"
    )
```

```
alpha 0.001  features kept 14 of 20  test R² 0.9594
alpha 0.01   features kept  5 of 20  test R² 0.9583
alpha 0.05   features kept  2 of 20  test R² 0.9537
alpha 0.2    features kept  1 of 20  test R² 0.9254
```

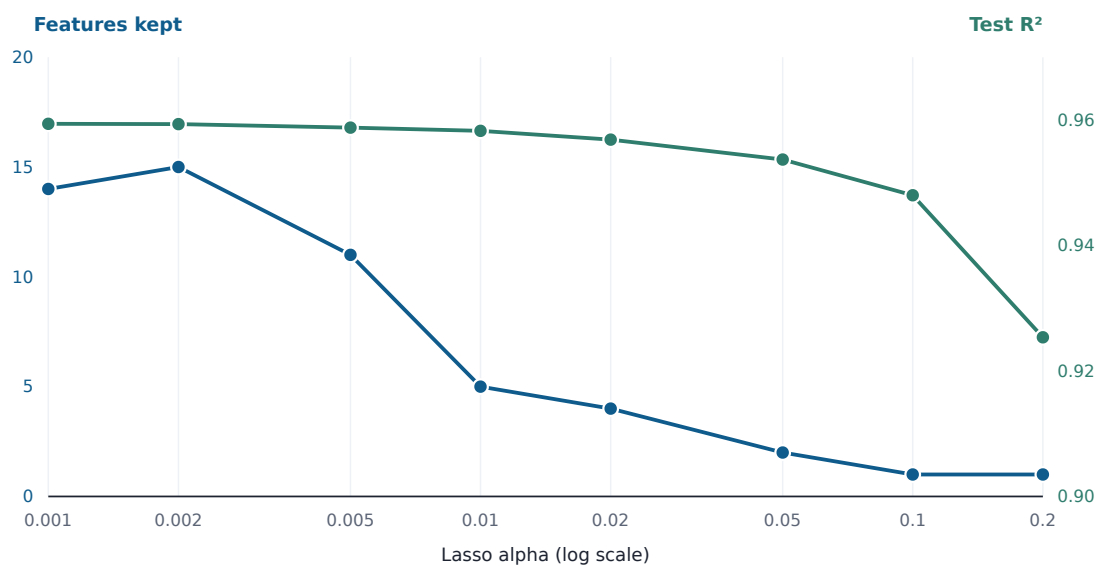


Figure 37.1 — Lasso as α grows. Five features give almost the same accuracy as fourteen; one feature (last year's revenue) still explains most of the variation.

What to tell Anita. "Next year's revenue for an account is mostly last year's revenue, adjusted up for hospitality accounts, accounts buying across several product categories, and down for accounts that pay late or complain." That sentence comes straight from the five features lasso keeps at $\alpha = 0.01$.

When regularization matters most: many features relative to rows (for example, 500 product-level features for 2,000 customers), correlated features, and any time you'd otherwise overfit. For logistic regression, scikit-learn regularizes by default (next section).

37.3 Logistic regression

How it works

Logistic regression predicts a **probability** for a yes-or-no outcome. It computes the same weighted sum as linear regression, called the **log-odds** or z , and passes it through the **sigmoid** function, which squeezes any number into the range 0 to 1:

$$z = b + w_1 \times \text{feature}_1 + \dots + w_n \times \text{feature}_n$$

$$\text{probability} = 1 \div (1 + e^{(-z)})$$

$z = 0$ gives 0.5; large positive z gives probabilities near 1; large negative z , near 0. The weights are chosen by **maximum likelihood**, which is the same as minimizing log loss (Chapter 35, section 35.9). There's no exact formula, so it's solved by an optimizer, a smarter relative of gradient descent.

Despite its name, it's a **classification** algorithm. Now the churn problem:

```

from sklearn.linear_model import LogisticRegression

def clf_pipeline(model, scale=True):
    num_steps = [("fill", SimpleImputer(strategy="median", add_indicator=True))]
    if scale:
        num_steps.append(("scale", StandardScaler()))
    prepare = ColumnTransformer(
        [
            ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
            ("num", Pipeline(num_steps), NUMS),
        ]
    )
    return Pipeline([("prepare", prepare), ("model", model)])

X_tr, y_tr = train_acc[CATS + NUMS], train_acc["churned_2025"]
X_va, y_va = valid_acc[CATS + NUMS], valid_acc["churned_2025"]
results = {}

def clf_report(name, pipe):
    pipe.fit(X_tr, y_tr)
    p = pipe.predict_proba(X_va)[ :, 1]
    results[name] = roc_auc_score(y_va, p)
    print(f"{name:<30} AUC {results[name]:.3f}    log loss {log_loss(y_va, p):.4f}")
    return pipe

logit = clf_report(
    "logistic regression", clf_pipeline(LogisticRegression(max_iter=2000))
)
coef = pd.Series(
    logit.named_steps["model"].coef_[0],
    index=logit.named_steps["prepare"].get_feature_names_out(),
)
odds = np.exp(coef).round(2).sort_values()
print("odds ratios (per 1 standard deviation for numbers):")
print(pd.concat([odds.head(3), odds.tail(4)]).to_string())

```

```

logistic regression          AUC 0.764    log loss 0.2833
odds ratios (per 1 standard deviation for numbers):
num__categories_bought      0.57
num__units_2024             0.66
cat__segment_Wholesale      0.74
num__complaints_2024        1.30
num__revenue_2024           1.31
num__late_payment_days      1.53
num__days_since_last_order  2.10

```

How it works: `np.exp(coef)` turns each weight into an **odds ratio**: how the odds of churning are multiplied when the feature rises by one unit (here, one standard deviation, since numbers are scaled; for a one-hot column, when the category applies).

Reading it. On the validation set, logistic regression reaches an AUC of 0.764. The odds ratios read like a sales manager's instincts:

- **days_since_last_order 2.10:** one standard deviation longer without an order (about 80 days) doubles the odds of churning. The strongest signal.

- **late_payment_days 1.53** and **complaints_2024 1.30**: late payers and complainers are more likely to leave.
- **categories_bought 0.57**: each extra standard deviation of product breadth (about one more category) cuts the odds of churning by 43%. Customers who buy several product lines are stickier.

revenue_2024 1.31 and **units_2024 0.66** point in opposite directions. They're highly correlated, so the model balances one against the other; read them together, not separately (the same multicollinearity as in section 37.1).

The sigmoid by hand, and the setting C

Take the first validation account. The model's weighted sum for it is $z = -2.124$:

$$\text{probability} = 1 \div (1 + e^{2.124}) = 1 \div (1 + 8.365) = 1 \div 9.365 = \mathbf{0.1068}$$

```
one = X_va.iloc[[0]]
z = logit.decision_function(one)[0]
print(f"z = {z:.3f}    sigmoid(z) = 1 / (1 + e^-z) = {1 / (1 + np.exp(-z)):.4f}")
print(f"predict_proba: {logit.predict_proba(one)[0, 1]:.4f}")
for C in [0.01, 0.1, 1, 10]:
    clf_report(f"logistic, C={C}", clf_pipeline(LogisticRegression(C=C, max_iter=2000)))
```

```
z = -2.124    sigmoid(z) = 1 / (1 + e^-z) = 0.1068
predict_proba: 0.1068
logistic, C=0.01          AUC 0.762    log loss 0.2778
logistic, C=0.1           AUC 0.764    log loss 0.2821
logistic, C=1             AUC 0.764    log loss 0.2833
logistic, C=10            AUC 0.765    log loss 0.2831
```

The hand calculation matches `predict_proba` exactly. ✓

C is logistic regression's regularization setting, and it works backwards from α : **smaller C means stronger regularization**. `C=1` is the default, so scikit-learn's logistic regression is regularized (with the ridge-style L2 penalty) unless you change it. Here, C barely matters: AUC stays between 0.762 and 0.765. As with linear regression, 3,000 rows and 20 features leave little to regularize.

When to use logistic regression: almost always as your first real model. It trains in milliseconds, gives calibrated probabilities (usually), explains itself through odds ratios, and is hard to beat when effects are roughly additive, as Chapter 36's leads were. Its weakness is **interactions and thresholds**: it can't see "late payments matter only for small retailers" unless you build that feature. Section 37.10 shows how much that costs here, and how to fix it.

37.4 k-nearest neighbors

How it works

k-nearest neighbors (k-NN) doesn't learn weights at all. To predict for a new account, it finds the k most similar accounts in the training data, measured by Euclidean distance (Chapter 35), and predicts the share of those neighbors that churned. "Training" means storing the data.

That makes two things critical: **the choice of k** (few neighbors gives noisy predictions; many gives blurry ones) and **scaling** (distance is meaningless across unscaled units).

```
from sklearn.neighbors import KNeighborsClassifier

for k in [5, 25, 75]:
    clf_report(f"k-NN, k={k}", clf_pipeline(KNeighborsClassifier(n_neighbors=k)))
clf_report(
    "k-NN, k=25, NOT scaled",
    clf_pipeline(KNeighborsClassifier(n_neighbors=25), scale=False),
)
```

k-NN, k=5	AUC 0.684	log loss 1.5887
k-NN, k=25	AUC 0.765	log loss 0.5255
k-NN, k=75	AUC 0.775	log loss 0.3057
k-NN, k=25, NOT scaled	AUC 0.633	log loss 0.4735

Reading it.

- **$k = 5$** gives an AUC of 0.684 and a terrible log loss (1.59). With five neighbors, the only possible probabilities are 0, 0.2, 0.4, 0.6, 0.8, and 1, and a confident 0 for an account that churns is punished severely (Chapter 35).
- **$k = 25$ and 75** do much better (0.765 and 0.775), close to logistic regression.
- **Without scaling**, AUC collapses to 0.633. Revenue, measured in rupees, dominates every distance, so the "nearest" accounts are the ones with similar revenue, whatever their churn signals.

When to use k-NN: small datasets with few, meaningful, scaled features; as a quick baseline; and inside recommenders (Chapter 42 uses the same "nearest neighbors" idea on products). Avoid it with many features (in high dimensions, every point is roughly equally far from every other, the **curse of dimensionality**), with large datasets (prediction must search all training rows), and when you need to explain a decision.

37.5 Naive Bayes

How it works

Naive Bayes applies **Bayes' rule** to classification. For each class (churned, stayed), it multiplies:

the **prior**: how common the class is $\cdot$ the **likelihood** of each feature value, given the class

and compares the results. It's "naive" because it assumes features are **independent** given the class, so their likelihoods can be multiplied. That's almost never true, and the method often works anyway.

A worked example

Use two yes/no signals from the training accounts: *late-paying small retailer* (more than 25 days late on average, size Small, segment Retail) and *no order in the last 90 days*.

```
late_small_retail = (
    (train_acc["late_payment_days"] > 25)
    & (train_acc["company_size"] == "Small")
    & (train_acc["segment"] == "Retail")
)
recent = train_acc["days_since_last_order"] <= 90
table = pd.crosstab(
    [
        late_small_retail.rename("late small retail"),
        recent.rename("ordered in last 90 days"),
    ],
    train_acc["churned_2025"].rename("churned"),
)
print(table)
counts = train_acc["churned_2025"].value_counts()
print("class counts:", counts.to_dict())
for flag, name in [
    (late_small_retail, "late small retail"),
    (~recent, "no order in 90 days"),
]:
    print(
        f"P({name} | churned) = {flag[train_acc['churned_2025'] == 1].mean():.3f}    "
        f"P({name} | stayed) = {flag[train_acc['churned_2025'] == 0].mean():.3f}"
    )
```

```
churned
late small retail ordered in last 90 days
False      False      421  145
           True      2215  85
True       False      10  40
           True       64  20
class counts: {0: 2710, 1: 290}
P(late small retail | churned) = 0.207   P(late small retail | stayed) = 0.027
P(no order in 90 days | churned) = 0.638   P(no order in 90 days | stayed) = 0.159
```

For an account that is a late-paying small retailer **and** has had no order in 90 days:

churned: prior $290 \div 3,000 = 0.0967 \times P(\text{late small retail} \mid \text{churned}) 0.207 \times P(\text{no order in 90 days} \mid \text{churned}) 0.638 = \mathbf{0.01276}$

stayed: prior $2,710 \div 3,000 = 0.9033 \times 0.027 \times 0.159 = \mathbf{0.00388}$

$P(\text{churned}) = 0.01276 \div (0.01276 + 0.00388) = \mathbf{0.767}$

Naive Bayes says 76.7% (76.5% with the unrounded likelihoods). Check it against the table: training accounts with both signals churned 40 times out of 50, **80%**. The estimate is close, because these two signals really are roughly independent here. When features are strongly related (revenue, units, and orders, for example), multiplying their likelihoods counts the same evidence several times, and the probabilities become overconfident.

Now the full model. `GaussianNB` assumes each numeric feature follows a normal distribution within each class:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import FunctionTransformer

to_dense = FunctionTransformer(lambda X: X.toarray() if hasattr(X, "toarray") else X)
nb = clf_pipeline(GaussianNB())
nb.steps.insert(1, ("dense", to_dense))
clf_report("Gaussian Naive Bayes", nb)
```

Gaussian Naive Bayes	AUC 0.740	log loss 1.2866
----------------------	-----------	-----------------

How it works: `GaussianNB` can't read the sparse one-hot output, so a `FunctionTransformer` converts it to a normal (dense) array, inserted as an extra pipeline step. `nb.steps.insert(1, ...)` puts it between preparation and the model.

Reading it. AUC 0.740, a little below logistic regression, and a log loss of 1.29, far worse. Naive Bayes ranks reasonably but its probabilities are overconfident: the independence assumption, applied to 25 correlated columns, piles up evidence. Chapter 35 warned that skewed features like revenue aren't normal either.

When to use Naive Bayes: text classification (each word as a feature; Chapter 41 uses it for support tickets), very small datasets, and when you need something fast and simple. Don't use its probabilities without calibration (Chapter 39).

37.6 Decision trees

How it works

A decision tree asks a sequence of yes/no questions, each about one feature: "Is *days since last order more than 160*?" Every question splits the rows into two groups. Rows that end in the same final group, a **leaf**, get the same prediction: that leaf's churn rate.

To choose each question, the tree tries **every feature and every threshold** and picks the split that makes the groups purest. Purity is measured by **Gini impurity** or **entropy** (Chapter 35):

$$\text{Gini} = 1 - p^2 - (1 - p)^2 = 2 \times p \times (1 - p), \text{ where } p \text{ is the group's churn rate}$$

$$\text{entropy} = -(p \times \log_2 p + (1 - p) \times \log_2 (1 - p))$$

Both are 0 for a pure group and highest at $p = 0.5$.

A split by hand

Compare two candidate first questions on the 3,000 training accounts, 290 of which churned ($p = 0.0967$):

Gini before any split = $2 \times 0.0967 \times 0.9033 = \mathbf{0.1746}$

Split A: days since last order > 90. 616 accounts say yes, with a 30.0% churn rate; 2,384 say no, with 4.4%.

Gini(yes) = $2 \times 0.300 \times 0.700 = 0.4200$ Gini(no) = $2 \times 0.044 \times 0.956 = 0.0841$

weighted Gini after = $(616 \div 3,000) \times 0.4200 + (2,384 \div 3,000) \times 0.0841 = 0.0862 + 0.0669 = \mathbf{0.1531}$

Split B: late payment days > 25. Yes: 387 accounts, 23.0% churn. No: 2,613, 7.7%.

weighted Gini after = $(387 \div 3,000) \times (2 \times 0.230 \times 0.770) + (2,613 \div 3,000) \times (2 \times 0.077 \times 0.923) = 0.0457 + 0.1238 = \mathbf{0.1695}$

Split A reduces impurity from 0.1746 to 0.1531; split B only to 0.1695. The tree prefers A. Using entropy gives the same ranking (information gain, Chapter 35):

```
def gini(y):
    p = np.mean(y)
    return 1 - p**2 - (1 - p) ** 2

def entropy(y):
    p = np.mean(y)
    return 0.0 if p in (0, 1) else -(p * np.log2(p) + (1 - p) * np.log2(1 - p))

y_all = train_acc["churned_2025"]
for rule_name, rule in [
    ("days_since_last_order > 90", train_acc["days_since_last_order"] > 90),
    ("late_payment_days > 25", train_acc["late_payment_days"] > 25),
]:
    left, right = y_all[rule], y_all[~rule]
    w_l, w_r = len(left) / len(y_all), len(right) / len(y_all)
    print(
        f"{rule_name}: yes {len(left)} ({left.mean():.1%} churn),"
        f" no {len(right)} ({right.mean():.1%})"
    )
    print(
        f"    Gini {gini(y_all):.4f} -> {w_l * gini(left) + w_r * gini(right):.4f}    "
        f"entropy {entropy(y_all):.4f} -> {w_l * entropy(left) + w_r * entropy(right):.4f}"
    )
```

```
days_since_last_order > 90: yes 616 (30.0% churn), no 2384 (4.4%)
    Gini 0.1746 -> 0.1532    entropy 0.4583 -> 0.3881
late_payment_days > 25: yes 387 (23.0% churn), no 2613 (7.7%)
    Gini 0.1746 -> 0.1694    entropy 0.4583 -> 0.4411
```

(The code's weighted values, 0.1532 and 0.1694, differ from the hand values in the last digit because the churn rates were rounded to three decimals by hand.)

Depth, and overfitting

Keep splitting and every leaf eventually holds one account, and the tree memorizes the training data. Watch it happen:

```
from sklearn.tree import DecisionTreeClassifier, export_text

for depth in [2, 4, 6, 10, None]:
    tree = clf_pipeline(
        DecisionTreeClassifier(max_depth=depth, random_state=37), scale=False
    ).fit(X_tr, y_tr)
    tr_auc = roc_auc_score(y_tr, tree.predict_proba(X_tr)[: , 1])
    va_auc = roc_auc_score(y_va, tree.predict_proba(X_va)[: , 1])
    print(
        f"max_depth {str(depth):<5} train AUC {tr_auc:.3f}    valid AUC {va_auc:.3f}    "
        f"leaves {tree.named_steps['model'].get_n_leaves()}"
    )

small_tree = clf_pipeline(
    DecisionTreeClassifier(max_depth=2, random_state=37), scale=False
).fit(X_tr, y_tr)
print(
    export_text(
        small_tree.named_steps["model"],
        feature_names=list(small_tree.named_steps["prepare"].get_feature_names_out()),
        show_weights=True,
    )
)

clf_report(
    "decision tree, depth 4",
    clf_pipeline(
        DecisionTreeClassifier(max_depth=4, min_samples_leaf=20, random_state=37),
        scale=False,
    ),
)
```

```

max_depth 2      train AUC 0.759   valid AUC 0.708   leaves 4
max_depth 4      train AUC 0.829   valid AUC 0.748   leaves 16
max_depth 6      train AUC 0.886   valid AUC 0.733   leaves 48
max_depth 10     train AUC 0.969   valid AUC 0.609   leaves 161
max_depth None   train AUC 1.000   valid AUC 0.557   leaves 277
|--- num_days_since_last_order <= 160.50
| |--- num_days_since_last_order <= 94.50
| | |--- weights: [2305.00, 106.00] class: 0
| |--- num_days_since_last_order > 94.50
| | |--- weights: [232.00, 52.00] class: 0
|--- num_days_since_last_order > 160.50
| |--- num_late_payment_days <= 25.50
| | |--- weights: [160.00, 93.00] class: 0
| |--- num_late_payment_days > 25.50
| | |--- weights: [13.00, 39.00] class: 1

decision tree, depth 4          AUC 0.758   log loss 0.2810

```

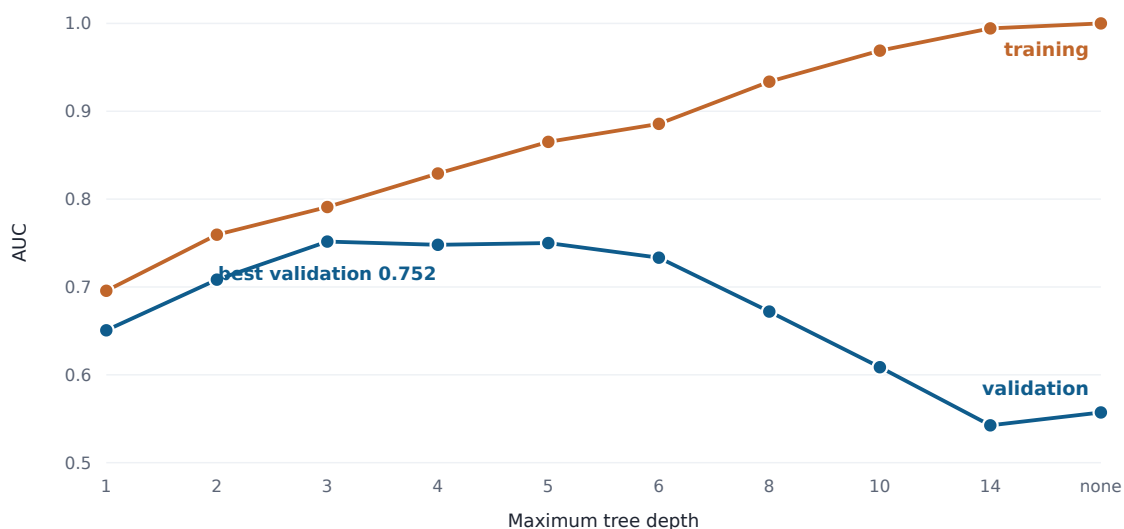


Figure 37.2 — A decision tree overfitting. Deeper trees fit the training data better and better, while their performance on new accounts peaks early and then collapses.

Reading it.

- A tree with no depth limit has **277 leaves** and a **perfect** training AUC of 1.000, and a validation AUC of 0.557, barely better than a coin. It memorized noise.
- Depth 4 is the best of the depths printed here (0.748); Figure 37.2, which tries more depths, finds depth 3 slightly better (0.752). The printed depth-2 tree is readable by anyone: accounts with no order for more than 160 days **and** more than 25 days of late payments churned 39 times out of 52. That's the planted interaction, found automatically.
- The tree found the interaction logistic regression couldn't see, but a single tree is **unstable**: small changes in the data change the first split and everything below it. Its best validation AUC is below logistic regression's.

Key settings: `max_depth`, `min_samples_leaf` (the fewest rows a leaf may hold; 20 to 100 is a sensible range for thousands of rows), and `criterion` ("gini" or "entropy"; they rarely differ much).

When to use a single tree: when a person must be able to follow the rules (a credit policy, a triage checklist), and as the building block for the next two methods. Trees need no scaling and handle interactions and thresholds naturally.

37.7 Random forests

How it works

A random forest trains **hundreds of trees** and averages their predictions. Averaging only helps if the trees make *different* mistakes, so each tree is made different in two ways:

1. **Bagging** (bootstrap aggregating): each tree trains on a random sample of the training rows, drawn with replacement.
2. **Random features:** at each split, each tree considers only a random subset of the features.

Each tree overfits in its own way; the average cancels much of the noise and keeps the signal. It's the committee from *In plain English*.

```
from sklearn.ensemble import RandomForestClassifier

forest = clf_report(
    "random forest",
    clf_pipeline(
        RandomForestClassifier(n_estimators=300, min_samples_leaf=5, random_state=37),
        scale=False,
    ),
)
imp = pd.Series(
    forest.named_steps["model"].feature_importances_,
    index=forest.named_steps["prepare"].get_feature_names_out(),
)
print(imp.sort_values(ascending=False).head(6).round(3).to_string())
```

random forest	AUC 0.785	log loss 0.2675
num_days_since_last_order	0.295	
num_late_payment_days	0.112	
num_revenue_2024	0.088	
num_units_2024	0.084	
num_tenure_months	0.070	
num_avg_discount_pct	0.066	

Reading it. AUC 0.785, the best so far and above logistic regression's 0.764. The forest can use thresholds and interactions without being told about them.

Feature importance. `feature_importances_` shows how much each feature reduced impurity across all trees. `days_since_last_order` leads by far, then `late_payment_days`. Two cautions: impurity-based importance is inflated for features with many distinct values (revenue has thousands; segment has three), and it says nothing about direction. Chapter 39 uses **permutation importance** and **SHAP**, which are more reliable.

Key settings: `n_estimators` (more trees never overfit, only cost time; 200–500 is usually plenty), `min_samples_leaf` or `max_depth` (limit the individual trees), and `max_features` (how many features each split considers; the default for classification is the square root of the number of features).

When to use a random forest: a strong, forgiving default for tabular data. It needs little tuning, rarely overfits badly, handles mixed feature types, and gives a quick importance ranking. It's usually a little behind well-tuned gradient boosting, and its models are large.

37.8 Gradient boosting

How it works

A forest builds trees **independently** and averages them. **Gradient boosting** builds them **one after another**, each new tree correcting the errors of the model so far:

1. Start with a simple prediction for everyone, such as the average.
2. Compute each row's **residual**: how far off the current prediction is. (More precisely, the negative gradient of the loss, Chapter 35. For squared error, that's the plain residual.)
3. Fit a **small tree** to the residuals.
4. Add that tree's predictions to the model, **scaled down** by a **learning rate**.
5. Repeat for hundreds of rounds.

Three rounds by hand

Take 8 training accounts, one feature (days since last order), and squared error, with a learning rate of 0.5 and one-split trees (**stumps**):

```
from sklearn.tree import DecisionTreeRegressor

toy = train_acc.head(8)
x_toy = toy[["days_since_last_order"]].to_numpy()
y_toy = toy["churned_2025"].to_numpy().astype(float)
pred = np.full(len(y_toy), y_toy.mean())
print("targets:      ", y_toy)
print(
    f"round 0: predict {y_toy.mean():.3f} for all"
    f"    squared error {np.mean((y_toy - pred) ** 2):.4f}"
)
for r in range(1, 4):
    residual = y_toy - pred
    stump = DecisionTreeRegressor(max_depth=1, random_state=37).fit(x_toy, residual)
    pred = pred + 0.5 * stump.predict(x_toy)
    print(
        f"round {r}: split at days <= {stump.tree_.threshold[0]:.1f}    "
        f"squared error {np.mean((y_toy - pred) ** 2):.4f}"
    )
print("final:      ", pred.round(3))
```

```

targets:      [0. 0. 1. 0. 0. 0. 0. 0.]
round 0: predict 0.125 for all    squared error 0.1094
round 1: split at days <= 77.5   squared error 0.0977
round 2: split at days <= 42.0   squared error 0.0818
round 3: split at days <= 77.5   squared error 0.0713
final:        [0.059 0.059 0.303 0.153 0.153 0.059 0.153 0.059]

```

Reading it. One of the eight accounts churned. Round 0 predicts the average, 0.125, for everyone. Each round fits a stump to what's still wrong, adds half of its correction, and the error falls: 0.1094, 0.0977, 0.0818, 0.0713. After three rounds, the churner's predicted value is 0.303, well above the rest. Real libraries do exactly this with log loss instead of squared error, deeper trees, and hundreds of rounds.

The libraries

Four implementations matter in practice:

Library	Strengths	Notes
scikit-learn HistGradientBoostingClassifier	Already installed; fast on medium data; handles missing values	Good default; fewer options
XGBoost	Mature, widely used, many options, GPU support	Industry standard in competitions and production
LightGBM	Very fast on large data; grows trees leaf-wise	num_leaves is its key setting; can overfit small data
CatBoost	Handles categorical columns natively and carefully; strong defaults	Slower to train; less tuning needed

The settings that matter most are shared across all four, under slightly different names: **number of trees** (max_iter, n_estimators, iterations), **learning rate** (smaller needs more trees but generalizes better; 0.01–0.1 is typical), **tree size** (max_depth, num_leaves), **minimum rows per leaf** (min_samples_leaf, min_child_samples), **row and column sampling** (subsample, colsample_bytree), and **L2 regularization**.

```

import catboost
import lightgbm
import xgboost
from sklearn.ensemble import HistGradientBoostingClassifier

print(
    "versions: xgboost",
    xgboost.__version__,
    "| lightgbm",
    lightgbm.__version__,
    "| catboost",
    catboost.__version__,
)
clf_report(
    "HistGradientBoosting default",
    clf_pipeline(HistGradientBoostingClassifier(random_state=37), scale=False),
)
clf_report(
    "HistGradientBoosting tuned",
    clf_pipeline(
        HistGradientBoostingClassifier(
            learning_rate=0.05,
            max_depth=3,
            min_samples_leaf=40,
            max_iter=300,
            random_state=37,
        ),
        scale=False,
    ),
)
clf_report(
    "XGBoost",
    clf_pipeline(
        xgboost.XGBClassifier(
            n_estimators=300,
            learning_rate=0.05,
            max_depth=3,
            subsample=0.8,
            colsample_bytree=0.8,
            random_state=37,
            n_jobs=1,
        ),
        scale=False,
    ),
)
clf_report(
    "LightGBM",
    clf_pipeline(
        lightgbm.LGBMClassifier(
            n_estimators=300,
            learning_rate=0.05,
            num_leaves=8,
            min_child_samples=40,
            subsample=0.8,
            subsample_freq=1,
            colsample_bytree=0.8,
            random_state=37,
            verbose=-1,
        ),
        scale=False,
    ),
)

cat_model = catboost.CatBoostClassifier(
    iterations=500,

```

```

learning_rate=0.05,
depth=4,
random_seed=37,
verbose=0,
thread_count=1,
)
cat_model.fit(X_tr, y_tr, cat_features=CATS)
p_cat = cat_model.predict_proba(X_va)[ :, 1]
results["CatBoost"] = roc_auc_score(y_va, p_cat)
print(
    f"'CatBoost (native categories)':<30} AUC {results['CatBoost']:.3f}"
    f"    log loss {log_loss(y_va, p_cat):.4f}"
)

```

```

versions: xgboost 3.4.1 | lightgbm 4.7.0 | catboost 1.2.10
HistGradientBoosting default    AUC 0.780    log loss 0.3151
HistGradientBoosting tuned      AUC 0.788    log loss 0.2731
XGBoost                          AUC 0.793    log loss 0.2700
LightGBM                        AUC 0.780    log loss 0.2794
CatBoost (native categories)    AUC 0.796    log loss 0.2710

```

How it works: XGBoost and LightGBM follow scikit-learn's `fit` and `predict_proba` interface, so they drop straight into the pipeline. CatBoost is given the raw columns and told which are categorical (`cat_features=CATS`); it encodes them itself, using a carefully ordered form of target encoding that avoids the leak from Chapter 36. On this run, CatBoost took about 1.7 seconds and the others 0.1–0.2 seconds each.

Reading it. All the boosting models beat logistic regression on validation (0.780–0.796 against 0.764). Default and tuned settings differ, and so do the libraries, by a couple of points. **With 1,000 validation accounts and 97 churners, differences of 0.01 are within noise.** Don't pick XGBoost over LightGBM on this evidence; section 37.10 uses cross-validation to compare properly.

When to use gradient boosting: the default choice for tabular business data when accuracy matters and you have at least a few thousand rows. It handles thresholds, interactions, missing values, and mixed features. Its costs: more settings to tune, slower training than linear models, overfitting on small data, and less transparency (Chapter 39's SHAP values help).

37.9 Support vector machines

How it works

A support vector machine (SVM) looks for the boundary between classes with the **widest margin**: the biggest gap to the nearest points on either side. Those nearest points, the **support vectors**, alone define the boundary. The setting **C** trades a wide margin against misclassified training points (as with logistic regression, smaller C means more regularization).

For boundaries that aren't straight lines, the **kernel trick** measures similarity between rows in a way that corresponds to a curved boundary, without computing new features explicitly. The **RBF** (radial basis function) kernel is the usual choice.

```

from sklearn.svm import SVC

clf_report(
    "SVM, linear kernel",
    clf_pipeline(SVC(kernel="linear", probability=True, random_state=37)),
)
clf_report(
    "SVM, linear, balanced",
    clf_pipeline(
        SVC(kernel="linear", class_weight="balanced", probability=True, random_state=37)
    ),
)
clf_report(
    "SVM, RBF, balanced",
    clf_pipeline(
        SVC(
            kernel="rbf",
            C=1.0,
            class_weight="balanced",
            probability=True,
            random_state=37,
        )
    ),
)

```

SVM, linear kernel	AUC 0.536	log loss 0.3337
SVM, linear, balanced	AUC 0.771	log loss 0.2787
SVM, RBF, balanced	AUC 0.780	log loss 0.2711

Reading it.

- The **linear SVM with default settings** scores an AUC of 0.536, almost a coin toss. With only 9.7% churners, the widest-margin solution is to push nearly all accounts to the "stayed" side, and the ranking within that side is nearly meaningless.
- **class_weight="balanced"** makes each churner count about nine times as much in the loss, so the boundary takes churners seriously. AUC jumps to 0.771. Chapter 39 covers class weights and other ways to handle imbalanced data.
- The **RBF kernel**, balanced, reaches 0.780, similar to the forest.

How it works: SVMs produce a distance from the boundary, not a probability. `probability=True` adds an extra calibration step (Platt scaling) with internal cross-validation, which slows training.

When to use SVMs: medium-sized datasets (up to tens of thousands of rows) with many features, such as text, and problems where the classes are well separated. Training time grows quickly with rows, they need scaling, and they don't give probabilities or explanations naturally. For typical business tables, gradient boosting has largely replaced them.

37.10 Bias, variance, and learning curves

The trade-off

A model's error on new data comes from two sources, beyond pure chance:

- **Bias:** the model is too simple to capture the pattern. A straight line can't follow a threshold. Symptom: training and validation scores are both mediocre and close together. **Underfitting.**
- **Variance:** the model is so flexible that it fits noise in the training sample. Symptom: training score much higher than validation. **Overfitting.** The unlimited decision tree was the extreme case.

The fix depends on which you have. For **high bias**, add features, use a more flexible model, or reduce regularization. For **high variance**, get more data, simplify the model, increase regularization, or average many models (a forest).

Compare properly first

Validation scores on 1,000 accounts are noisy. Before diagnosing, compare the three strongest candidates with 5-fold cross-validation on all 4,000 non-test accounts:

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=37)
candidates = {
    "logistic regression": clf_pipeline(LogisticRegression(max_iter=2000)),
    "random forest": clf_pipeline(
        RandomForestClassifier(n_estimators=300, min_samples_leaf=5, random_state=37),
        scale=False,
    ),
    "HistGradientBoosting tuned": clf_pipeline(
        HistGradientBoostingClassifier(
            learning_rate=0.05,
            max_depth=3,
            min_samples_leaf=40,
            max_iter=300,
            random_state=37,
        ),
        scale=False,
    ),
}
X_rest, y_rest = rest[CATS + NUMS], rest["churned_2025"]
for name, pipe in candidates.items():
    s = cross_val_score(pipe, X_rest, y_rest, cv=folds, scoring="roc_auc")
    print(f"{name:<28} mean AUC {s.mean():.3f}    sd {s.std():.3f}    folds {s.round(3)}")
```

logistic regression	mean AUC	0.807	sd	0.007	folds	[0.812 0.796 0.811 0.803 0.815]
random forest	mean AUC	0.826	sd	0.020	folds	[0.862 0.818 0.822 0.803 0.824]
HistGradientBoosting tuned	mean AUC	0.836	sd	0.018	folds	[0.868 0.828 0.838 0.817 0.829]

On 4,000 accounts, the picture is clearer: **tuned boosting 0.836, random forest 0.826, logistic regression 0.807**, and the fold spreads (0.007–0.020) are smaller than the gaps. Boosting's advantage over logistic regression is real on this data.

Learning curves

A **learning curve** trains a model on increasing amounts of data and plots training and cross-validation scores for each size:

```
from sklearn.model_selection import learning_curve

sizes = [0.1, 0.25, 0.5, 0.75, 1.0]
curves = {}
for name in ["logistic regression", "HistGradientBoosting tuned"]:
    n, tr_s, va_s = learning_curve(
        candidates[name],
        X_rest,
        y_rest,
        cv=folds,
        scoring="roc_auc",
        train_sizes=sizes,
        shuffle=True,
        random_state=37,
    )
    curves[name] = (n, tr_s.mean(axis=1), va_s.mean(axis=1))
for size, a, b in zip(n, tr_s.mean(axis=1), va_s.mean(axis=1)):
    print(
        f"{name:<27} {size:>5} rows   train AUC {a:.3f}"
        f"      CV AUC {b:.3f}   gap {a - b:.3f}"
    )
```

logistic regression	320 rows	train AUC 0.870	CV AUC 0.760	gap 0.110
logistic regression	800 rows	train AUC 0.827	CV AUC 0.777	gap 0.051
logistic regression	1600 rows	train AUC 0.822	CV AUC 0.802	gap 0.020
logistic regression	2400 rows	train AUC 0.822	CV AUC 0.808	gap 0.015
logistic regression	3200 rows	train AUC 0.820	CV AUC 0.807	gap 0.013
HistGradientBoosting tuned	320 rows	train AUC 1.000	CV AUC 0.747	gap 0.253
HistGradientBoosting tuned	800 rows	train AUC 0.995	CV AUC 0.792	gap 0.203
HistGradientBoosting tuned	1600 rows	train AUC 0.979	CV AUC 0.805	gap 0.174
HistGradientBoosting tuned	2400 rows	train AUC 0.959	CV AUC 0.815	gap 0.144
HistGradientBoosting tuned	3200 rows	train AUC 0.945	CV AUC 0.836	gap 0.109

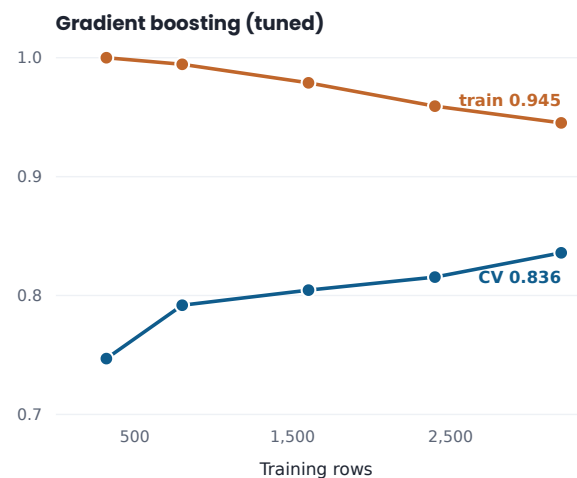
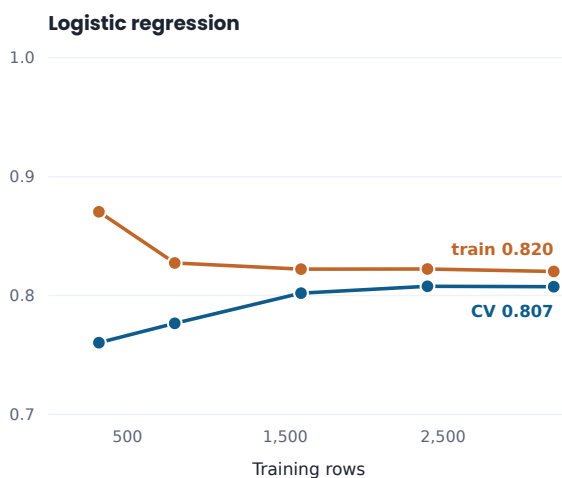


Figure 37.3 – Learning curves. Logistic regression has converged: more rows won't help, a sign of bias. Boosting still has a big train–CV gap and is still improving: a sign of variance that more data would reduce.

Reading it.

- **Logistic regression** has **high bias**. By 1,600 rows, its training and CV scores have nearly met (gap 0.013 at 3,200 rows), and the CV score has flattened at about 0.807. More data won't help. The model is missing something.
- **Boosting** has **higher variance**: a training AUC of 0.945 against 0.836 on CV. But its CV score is still climbing at 3,200 rows, so more accounts would likely help it further.

Fixing bias with features

Logistic regression's problem is structural: it can't represent "no order in 90 days" as a jump, or "late payments matter for small retailers" as an interaction. So build those as features, using what the tree in section 37.6 and the odds ratios suggested:

```
engineered = rest.copy()
test_eng = test_acc.copy()
for df in (engineered, test_eng):
    df["no_order_90"] = (df["days_since_last_order"] > 90).astype(int)
    df["no_order_180"] = (df["days_since_last_order"] > 180).astype(int)
    df["late_small_retail"] = (
        (df["late_payment_days"] > 25)
        & (df["company_size"] == "Small")
        & (df["segment"] == "Retail")
    ).astype(int)
    df["new_and_complaining"] = (
        (df["complaints_2024"] >= 2) & (df["tenure_months"] < 18)
    ).astype(int)
ENG = ["no_order_90", "no_order_180", "late_small_retail", "new_and_complaining"]
eng_pipe = Pipeline(
    [
        (
            "prepare",
            ColumnTransformer(
                [
                    ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
                    (
                        "num",
                        Pipeline(
                            [
                                (
                                    "fill",
                                    SimpleImputer(
                                        strategy="median", add_indicator=True
                                    ),
                                ),
                                ("scale", StandardScaler()),
                            ]
                        ),
                    ),
                ],
                NUMS + ENG,
            ),
        ),
        ("model", LogisticRegression(max_iter=2000)),
    ]
)
s = cross_val_score(
    eng_pipe, engineered[CATS + NUMS + ENG], y_rest, cv=folds, scoring="roc_auc"
)
print(f"logistic + 4 engineered features    mean AUC {s.mean():.3f}    sd {s.std():.3f}")
```

logistic + 4 engineered features mean AUC 0.849 sd 0.015

Reading it. With four engineered features, logistic regression's cross-validated AUC jumps from 0.807 to **0.849**, above tuned boosting's 0.836 and close to the 0.858 ceiling from section 37.0.

That's a useful lesson and an honest one, with a caveat. The features came from knowing where to look: thresholds and interactions that a tree and domain knowledge pointed to, and that match how this data was generated. On real data you won't know the true pattern, and boosting's advantage is that it finds such patterns without being told. The practical workflow is to use both: let boosting show what's possible, then ask whether a simpler model with good features gets close enough to be worth its transparency.

37.11 Hyperparameter tuning

Settings are chosen, not learned

Hyperparameters are the settings you choose before training: tree depth, learning rate, C , k . **Tuning** means trying different values and keeping the best, judged by cross-validation on the training data, never on the test set.

- **Grid search** (`GridSearchCV`) tries every combination of listed values. Five settings with four values each is $4^5 = 1,024$ combinations, times 5 folds: 5,120 fits. It grows too fast.
- **Random search** (`RandomizedSearchCV`) samples a fixed number of combinations from ranges you give. For the same budget, it explores more values of the settings that matter most. Use it instead of grid search by default.
- **Bayesian optimization** (Optuna, and others) chooses each next combination based on how earlier ones scored, spending more trials in promising regions.

Random search

```
from scipy.stats import loguniform, randint
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(
    clf_pipeline(HistGradientBoostingClassifier(random_state=37), scale=False),
    param_distributions={
        "model__learning_rate": loguniform(0.01, 0.3),
        "model__max_depth": [2, 3, 4, 6, None],
        "model__min_samples_leaf": randint(10, 100),
        "model__max_iter": randint(100, 500),
        "model__l2_regularization": loguniform(1e-3, 10),
    },
    n_iter=25,
    cv=folds,
    scoring="roc_auc",
    random_state=37,
    n_jobs=1,
)
search.fit(X_rest, y_rest)
print(f"best CV AUC {search.best_score_:.3f}")
print(
    {
        k.replace("model__", ""): (round(float(v), 4) if isinstance(v, float) else v)
        for k, v in search.best_params_.items()
    }
)
top = pd.DataFrame(search.cv_results_).sort_values("rank_test_score")
print(
    top[["mean_test_score", "std_test_score"]].head(5).round(3).to_string(index=False)
)
```

```
best CV AUC 0.841
{'l2_regularization': 2.7349, 'learning_rate': 0.0199, 'max_depth': 3, 'max_iter': 334,
 'min_samples_leaf': 93}
mean_test_score  std_test_score
           0.841           0.017
           0.837           0.017
           0.837           0.016
           0.836           0.016
           0.835           0.015
```

How it works:

- `param_distributions` gives a range or list for each setting. `loguniform(0.01, 0.3)` samples learning rates evenly on a log scale, so 0.01–0.03 gets as many tries as 0.1–0.3. Use it for any setting that spans orders of magnitude.
- The names use the pipeline step: `model__learning_rate` means "the `learning_rate` of the step named `model`". That's how a search reaches inside a pipeline.
- Every one of the 25 combinations is scored with the same 5 folds, and the preparation steps are re-learned in each fold: no leakage (Chapter 36). On this run, the search took about 40 seconds on one CPU core.

Reading it. The best combination scores 0.841, only 0.005 above the hand-picked settings used earlier (0.836), and the top five are all within 0.006 of each other, well inside their fold-to-fold standard deviation of about 0.017. **Tuning helped a little, and the exact winner is partly luck.** That's typical:

the first sensible settings get you most of the way. The best settings share a pattern worth noticing: a small learning rate, shallow trees (depth 3), large leaves (93 rows), and strong L2 regularization. With only 290 churners in training, the model does best when it's kept simple.

Optuna

```
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)

def objective(trial):
    model = HistGradientBoostingClassifier(
        learning_rate=trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
        max_depth=trial.suggest_int("max_depth", 2, 6),
        min_samples_leaf=trial.suggest_int("min_samples_leaf", 10, 100),
        max_iter=trial.suggest_int("max_iter", 100, 500),
        l2_regularization=trial.suggest_float("l2_regularization", 1e-3, 10, log=True),
        random_state=37,
    )
    return cross_val_score(
        clf_pipeline(model, scale=False), X_rest, y_rest, cv=folds, scoring="roc_auc"
    ).mean()

study = optuna.create_study(
    direction="maximize", sampler=optuna.samplers.TPESampler(seed=37)
)
study.optimize(objective, n_trials=25)
print(f"optuna {optuna.__version__}: best CV AUC {study.best_value:.3f}")
print(
    {
        k: round(v, 4) if isinstance(v, float) else v
        for k, v in study.best_params.items()
    }
)
```

```
optuna 5.0.0: best CV AUC 0.844
{'learning_rate': 0.0307, 'max_depth': 3, 'min_samples_leaf': 87, 'max_iter': 498,
 'l2_regularization': 5.3027}
```

How it works: Optuna calls `objective` repeatedly. Each `trial.suggest_...` call picks a value for one setting; the function returns the cross-validated score, and Optuna's default sampler (TPE, a Bayesian method) uses past results to choose the next values. `seed=37` makes the search repeatable.

Reading it. After the same 25 trials, Optuna finds 0.844, a little above random search. With larger search spaces or slower models, the advantage of choosing trials intelligently grows.

Watch out: tuning can overfit the validation data. Try enough combinations and one will score well by chance. Three habits keep you honest: tune with cross-validation, not one validation set; don't read much into differences smaller than the fold standard deviation; and score the **test set once**, at the end, for the final choice only.

The test set, once

Three candidates remain for churn: plain logistic regression, logistic regression with engineered features, and tuned boosting. Each is refitted on all 4,000 non-test accounts and scored once on the 1,000 test accounts:

```
final_models = {
    "logistic regression": clf_pipeline(LogisticRegression(max_iter=2000)),
    "logistic + engineered": eng_pipe,
    "tuned boosting (random search)": search.best_estimator_,
}
for name, pipe in final_models.items():
    cols = CATS + NUMS + (ENG if "engineered" in name else [])
    source = engineered if "engineered" in name else rest
    target = test_eng if "engineered" in name else test_acc
    pipe.fit(source[cols], source["churned_2025"])
    p = pipe.predict_proba(target[cols])[:, 1]
    print(
        f"{name:<32} test AUC {roc_auc_score(target['churned_2025'], p):.3f}    "
        f"log loss {log_loss(target['churned_2025'], p):.4f}"
    )
```

logistic regression	test AUC 0.797	log loss 0.2538
logistic + engineered	test AUC 0.850	log loss 0.2278
tuned boosting (random search)	test AUC 0.839	log loss 0.2375

Reading it. The test results agree with cross-validation: plain logistic regression **0.797**, tuned boosting **0.839**, logistic regression with engineered features **0.850**, also with the best log loss.

What to tell Anita. "We can rank accounts by their risk of stopping ordering well enough to act on. The strongest warning signs are no order in the last three months, late payments at small retail accounts, and complaints from newer accounts, and customers buying several product lines rarely leave. I recommend the simpler model built on those signs: it's slightly more accurate than the complex one on accounts it hadn't seen, and we can explain every score to the sales team."

37.12 Project result: lead scoring, baseline vs boosting

Now the question this chapter's project asks: does gradient boosting beat Chapter 36's logistic regression on Riverstone's leads? Tune boosting with a **time-series** cross-validation on the 2023–2024 leads (the leads arrive over time), then compare on validation and, once, on test:

```

import sys

sys.path.append(".") # so Python finds lead_data.py in this folder
from lead_data import LEAD_CATS, LEAD_NUMS, load_leads, make_model
from sklearn.model_selection import TimeSeriesSplit

leads_train, leads_valid, leads_test = load_leads()
LX = LEAD_CATS + LEAD_NUMS

lead_search = RandomizedSearchCV(
    make_model(LEAD_CATS, LEAD_NUMS, HistGradientBoostingClassifier(random_state=37)),
    param_distributions={
        "model__learning_rate": loguniform(0.01, 0.3),
        "model__max_depth": [2, 3, 4, 6, None],
        "model__min_samples_leaf": randint(20, 200),
        "model__max_iter": randint(100, 500),
        "model__l2_regularization": loguniform(1e-3, 10),
    },
    n_iter=20,
    cv=TimeSeriesSplit(n_splits=4),
    scoring="roc_auc",
    random_state=37,
    n_jobs=1,
)
lead_search.fit(leads_train[LX], leads_train["won"])
print(f"tuned boosting, time-series CV AUC {lead_search.best_score_:.3f}")

for name, fit_rows, eval_rows in [
    ("VALID", leads_train, leads_valid),
    ("TEST", pd.concat([leads_train, leads_valid]), leads_test),
]:
    base = make_model(LEAD_CATS, LEAD_NUMS).fit(fit_rows[LX], fit_rows["won"])
    boost = lead_search.best_estimator_.fit(fit_rows[LX], fit_rows["won"])
    for label, m in [("logistic (Ch 36)", base), ("tuned boosting", boost)]:
        p = m.predict_proba(eval_rows[LX])[:, 1]
        print(
            f"{name:<5} {label:<18} AUC {roc_auc_score(eval_rows['won'], p):.3f}  "
            f"log loss {log_loss(eval_rows['won'], p):.4f}"
        )

```

```

tuned boosting, time-series CV AUC 0.769
VALID logistic (Ch 36)    AUC 0.823    log loss 0.1948
VALID tuned boosting     AUC 0.829    log loss 0.1972
TEST  logistic (Ch 36)   AUC 0.838    log loss 0.2369
TEST  tuned boosting     AUC 0.832    log loss 0.2424

```

How it works: `lead_data.py` rebuilds Chapter 36's cleaned table and features in one call, so Chapter 36's steps aren't repeated here. `sys.path.append(".")` tells Python to look for it in the current folder (a notebook opened in `companion/ch37/` already does).

Reading it. On leads, **boosting doesn't win**. On validation it ranks marginally better (AUC 0.829 against 0.823) but its probabilities are worse (log loss 0.1972 against 0.1948). On the test set, logistic regression is ahead on both (0.838 against 0.832, and 0.2369 against 0.2424). The differences are small enough to be noise either way.

Why the opposite of the churn result? The leads data's signal is mostly **additive**: each feature (source, email type, text, quantity) pushes the odds up or down on its own, which is exactly the shape logistic regression assumes. There's little interaction for boosting to find, and with about 580 wins in training,

its extra flexibility mostly fits noise. The churn data has thresholds and interactions, and boosting's flexibility pays off.

The decision: keep logistic regression for lead scoring. It's as accurate, better calibrated, faster, and explainable. That's not a failure of the project; finding out that the simple model is enough is one of the most valuable results a data scientist can deliver.

Which algorithm when

Algorithm	Best for	Needs scaling?	Handles interactions?	Interpretable?	Watch out for
Linear regression	Numeric targets with roughly linear effects; explanation	Yes, for comparing or regularizing	No (build them)	Yes: coefficients	Outliers, multicollinearity, skewed targets
Ridge / lasso / elastic net	Many or correlated features; feature selection (lasso)	Yes	No	Yes	Choosing α ; lasso picking one of a correlated group
Logistic regression	First model for any classification; calibrated probabilities	Yes	No (build them)	Yes: odds ratios	Thresholds and interactions it can't see
k-nearest neighbors	Small data, few features; similarity lookups	Essential	Yes, implicitly	By example only	Many features, large data, choice of k
Naive Bayes	Text, very small data, speed	No (Gaussian: helps)	No	Partly	Overconfident probabilities
Decision tree	Rules people must follow; teaching	No	Yes	Yes, if shallow	Overfitting; instability
Random forest	Strong default with little tuning	No	Yes	Importance only	Large models; biased impurity importance
Gradient boosting	Best accuracy on tabular data with thousands of rows	No	Yes	With SHAP (Ch 39)	Tuning; overfitting small data
SVM	Medium data with many features (text); clear separation	Essential	Yes (kernels)	Little	Slow on large data; imbalance; no native probabilities

A default workflow for tabular problems: baselines (Chapter 36) → logistic or linear regression → a random forest or default gradient boosting → learning curves to diagnose → better features or tuned boosting, depending on what the curves say → cross-validated comparison → the simplest model that's close enough to the best → the test set, once.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Reading coefficients of correlated features one by one	Signs that make no sense (revenue up, units down)	Interpret groups together; use lasso or drop duplicates
Treating a coefficient as a cause	"Late payments cause 5% less growth" in a slide	Say "associated with"; use Chapter 31's methods for causes
Acting on a coefficient without an interval	Decisions based on noise (the inside-desk +3%)	Use <code>statsmodels</code> for confidence intervals (Chapter 22)
Regularizing unscaled features	Rupee-scale features escape the penalty	Standardize inside the pipeline
Confusing C and α	Increasing C and expecting more regularization	Smaller C = stronger regularization
k-NN or SVM without scaling	AUC far below simpler models	Scale in the pipeline
Trusting Naive Bayes probabilities	Log loss far worse than AUC suggests	Use it for ranking, or calibrate (Chapter 39)
Unlimited tree depth	Training AUC 1.0, validation near 0.5	Limit <code>max_depth</code> or <code>min_samples_leaf</code>
Default SVM on imbalanced data	AUC near 0.5	<code>class_weight="balanced"</code>
Choosing a library on one validation set	Switching libraries for 0.01 AUC	Cross-validate; differences within fold spread are noise
Grid search over many settings	Hours of compute, little gain	Random search or Optuna with a fixed budget
Tuning on the test set	Test score improves with each try	Tune with CV on training data; score test once
Assuming boosting always wins	Complex model deployed for no gain	Compare with a baseline and a linear model with good features
Impurity importance as the final word	High-cardinality features look most important	Permutation importance or SHAP (Chapter 39)

In the real world: "Should we buy the AutoML tool?"

In April 2026, Riverstone's finance team is renewing software licences, and Vikram forwards a quote for an **AutoML** platform, a service that tries dozens of algorithms and settings automatically. The vendor's demo, run on Riverstone's own account data, showed a "stacked ensemble of 40 models" scoring a churn AUC of 0.85. The question for Meera: *is it worth ₹6 lakh a year?*

Meera doesn't argue about AutoML in general. She sets up a fair comparison on the same test accounts, using this chapter's workflow.

Step 1: what does simple get? Logistic regression: 0.80. That's the floor any tool must beat.

Step 2: what does standard get? Gradient boosting with 25 rounds of random search, about a minute of compute: 0.84.

Step 3: what does thinking get? Logistic regression with four features built from the tree's first splits and the sales team's own warning signs (no order for three months, late-paying small retailers, new accounts with complaints): 0.85.

Step 4: what does each option cost to live with? The AutoML ensemble can't be explained account by account without extra tooling, needs the vendor's platform to run, and must be retrained through it. The logistic model is a few lines in the existing pipeline, and each account's score can be broken down into the signs that drove it, which the account managers will want before they call a customer.

Her note to Vikram:

"On our own test accounts, the AutoML ensemble (0.85) matches what we already get from a simple model built on the sales team's warning signs (0.85), and beats our quickly tuned boosting model (0.84) by a margin that's within the noise for 1,000 accounts. The simple model is free, runs in our existing pipeline, and can explain every score. I'd skip the licence for now. If we later have problems with many more features, like predicting demand for 200 products, AutoML would be worth a trial, because that's where automatic search saves real time."

Vikram cancels the renewal request. Two months later, Meera does use an open-source AutoML library for a weekend experiment on product demand (Chapter 40), as a quick way to see what's possible before building by hand, which is where such tools shine.

Tools

- **scikit-learn** (tested on 1.8.0; the current release at the time of writing is 1.9.1): `LinearRegression`, `RidgeCV`, `LassoCV`, `ElasticNetCV`, `LogisticRegression`, `KNeighborsClassifier`, `GaussianNB`, `DecisionTreeClassifier` and `export_text`, `RandomForestClassifier`, `HistGradientBoostingClassifier`, `SVC`, `learning_curve`, `RandomizedSearchCV`.
 - **XGBoost** 3.4.1, **LightGBM** 4.7.0, **CatBoost** 1.2.10, installed with `pip install xgboost lightgbm catboost`. All are free and open source; versions move quickly, so check each project's documentation for the current parameter names.
 - **Optuna** 5.0.0 (`pip install optuna`) for Bayesian hyperparameter search.
 - **SciPy** for `loguniform` and `randint` distributions in random search.
 - **statsmodels** for regression with confidence intervals and p-values (Chapter 22).
 - Everything ran on one CPU core, Python 3.12.3, on 17 September 2026. The slowest step, 25 rounds of 5-fold random search, took under a minute.
 - **Companion files:** `companion/generate_riverstone_accounts.py` (seed 20237) builds `companion/accounts/accounts.csv`; `companion/ch37/lead_data.py` rebuilds Chapter 36's lead table. Run the chapter's code from `companion/ch37/`. Data spec: `planning/data/riverstone-accounts.md`.
-

The project: lead scoring, baseline vs boosting

Goal: a documented, fair comparison of algorithms on one problem, ending in a justified choice. Section 37.12 ran the core comparison for leads; the project extends it and makes it yours.

Option A: your own data. Any classification or regression problem you set up with Chapter 36's workflow.

Option B: Riverstone. Lead scoring (`lead_data.py`) or account churn (`accounts.csv`), or both.

Steps:

1. **Start from the Chapter 36 pipeline and baselines.** Record the base rate, the rule of thumb, and logistic regression on validation.
2. **Train one model from each family:** logistic regression, k-NN, Naive Bayes, a decision tree, a random forest, one gradient-boosting library, and an SVM. Use the same pipeline and validation set for all.
3. **Hand-check one thing per family:** one sigmoid probability, one Naive Bayes posterior, one Gini split, three rounds of boosting on eight rows.
4. **Cross-validate the top three** (use a time-series split for leads) and report means and standard deviations.
5. **Draw learning curves** for the simplest and the most flexible of the three, and write two sentences diagnosing bias or variance.
6. **Try to close the gap** with features or with tuning, depending on the diagnosis. Use random search or Optuna with a fixed budget.
7. **Score the test set once** for your final two candidates.
8. **Write a one-page decision memo:** which model you recommend and why, what it costs to run and explain, and what you'd try next.

Stretch goals:

- Add **early stopping** to the gradient-boosting model (`early_stopping=True` in `HistGradientBoostingClassifier` , or an evaluation set in XGBoost and LightGBM) and compare training time and score.
- Build a **stacking** model (`StackingClassifier`) that combines logistic regression and boosting, and check fairly whether it beats either alone.
- Fit the regression model in `statsmodels` and compare its confidence interval for the inside-sales-desk coefficient with the effect you see.
- Try a **monotonic constraint** in `HistGradientBoostingClassifier` (for example, churn risk must not fall as days since last order rise) and see whether it costs any accuracy.

You've got it when...

- ☐ I can explain, in two sentences each, how linear regression, logistic regression, k-NN, Naive Bayes, trees, random forests, gradient boosting, and SVMs make a prediction.
 - ☐ I can read a regression coefficient and an odds ratio in plain language, and I know why correlated features make them unreliable.
 - ☐ I know what ridge, lasso, and elastic net penalize, why features must be scaled for them, and that smaller C means stronger regularization.
 - ☐ I can compute a sigmoid probability, a Naive Bayes posterior, and a Gini split by hand.
 - ☐ I can recognize an overfitting tree from its training and validation scores.
 - ☐ I can explain why averaging trees (forest) and chaining trees (boosting) both beat a single tree.
 - ☐ I know the main settings of gradient boosting and what each controls.
 - ☐ I handle class imbalance in SVMs and know when an algorithm needs scaling.
 - ☐ I can read learning curves and say whether a model suffers from bias or variance, and what to do about each.
 - ☐ I tune with random search or Optuna inside cross-validation, and I treat differences smaller than the fold spread as noise.
 - ☐ I compare complex models with baselines and with simple models that have good features, and I can defend choosing the simpler one.
-

Recap

- **Linear regression** fits a weighted sum by least squares; interpret coefficients carefully (scaling, multicollinearity, noise, not causes).
- **Ridge** shrinks all weights, **lasso** zeroes some (feature selection), **elastic net** mixes them. Scale first. They help most with many or correlated features.
- **Logistic regression** passes a weighted sum through the **sigmoid** to get a probability, fitted by maximum likelihood. Read it with **odds ratios**. Smaller **C** means more regularization. It can't see thresholds or interactions unless you build them.
- **k-NN** predicts from the nearest training rows; choose *k*, always scale, avoid many features.
- **Naive Bayes** multiplies a prior by per-feature likelihoods, assuming independence; fast, good for text, overconfident probabilities.
- **Decision trees** split on the question that most reduces **Gini** or **entropy**; readable, find interactions, overfit when deep.
- **Random forests** average many decorrelated trees (**bagging** + random features): a robust default.
- **Gradient boosting** adds small trees one at a time, each fitting the current errors, scaled by a **learning rate**: usually the most accurate on tabular data. XGBoost, LightGBM, and CatBoost are the main libraries.

- **SVMs** find the widest-margin boundary; kernels bend it; they need scaling and class weights for imbalance.
- **Bias** is underfitting, **variance** is overfitting; **learning curves** tell them apart.
- **Tune** with random search or **Optuna** inside cross-validation; score the test set once.
- The best algorithm depends on the data: boosting won on churn, tied on leads, and a simple model with good features beat both on churn.

Practice exercises

Code exercises run from `companion/ch37/` after the chapter's code (they use `train_acc`, `valid_acc`, `rest`, `test_acc`, `CATS`, `NUMS`, `clf_pipeline`, `clf_report`, `reg_pipeline`, `reg_train`, `reg_test`, `folds`, and the other names defined above). Predict each result before running it.

Warm-up

1. (*hand*) A logistic regression gives an account $z = 0.8$. What churn probability does it predict? What z corresponds to a probability of exactly 0.5?
2. (*hand*) An odds ratio for `complaints_2024` (unscaled, per complaint) is 1.45. An account with a 10% churn probability files one more complaint. What are its new odds and new probability?
3. (*hand*) A group of 200 accounts has 50 churners. Calculate its Gini impurity and its entropy.
4. For each situation, name the algorithm you'd try first and one reason: (a) 300 rows, 4 features, the sales team must follow the rules by hand; (b) 50,000 accounts, 60 mixed features, maximum accuracy; (c) classifying 20,000 support tickets by their words; (d) predicting next month's revenue per customer for a finance report that must explain each driver.

Core

5. Fit `RidgeCV` and `LassoCV` on the regression data **without** `log_revenue_2024`. How do R^2 and the coefficients of `log_units_2024` and `log_orders_2024` change? What does that tell you about multicollinearity?
6. For k-NN, try `weights="distance"` with $k = 75$. Does weighting closer neighbors more help on validation?
7. Using the training accounts, compute by hand (in code, without a tree) the weighted Gini after splitting on `categories_bought >= 3`. Is it a better first split than `days_since_last_order > 90`?
8. Train a random forest with `n_estimators` of 10, 50, 300, and 1,000 (keep `min_samples_leaf=5`). Report validation AUC. Does adding trees ever make it worse?
9. Continue the eight-row boosting example for 20 rounds instead of 3, and print the squared error every 5 rounds. What happens to the error, and why is that a warning sign rather than good news?
10. Draw a validation curve for `HistGradientBoostingClassifier`'s `max_depth` (2, 3, 4, 6, 8) using `validation_curve` with the 5 folds on `rest`. Where does it peak?

Stretch

- l1. Build a `StackingClassifier` from logistic regression and tuned boosting (`search.best_estimator_`), with a logistic regression on top. Cross-validate it on `rest`. Does it beat both parts?
- l2. Add the four engineered features from section 37.10 to the tuned boosting model. Does boosting gain as much as logistic regression did? Explain.
- l3. Using `statsmodels`, fit an ordinary least squares regression of `log_revenue_2025` on the same features (one-hot with drop first, unscaled numbers) and read the 95% confidence interval for the inside-sales-desk dummy. Does it include zero?

Think about it

- l4. A colleague says: "Boosting got 0.841 in tuning and logistic regression got 0.807, so boosting is 4% better." Give two corrections.
- l5. Why can a random forest's feature importance rank `revenue_2024` above `complaints_2024` even if complaints matter more for churn?
- l6. The sales head asks why the model was accurate last year but is worse this year. Using bias and variance, and Chapter 36's ideas, give three possible explanations.

Key terms

supervised learning · regression · classification · linear regression · least squares · intercept · coefficient · R^2 (coefficient of determination) · mean absolute error (MAE) · log-log model · dummy variable trap · multicollinearity · residual · regularization · penalty · alpha (α) · ridge regression (L2) · lasso (L1) · elastic net · feature selection · logistic regression · log-odds · sigmoid · odds ratio · C (inverse regularization) · k-nearest neighbors · curse of dimensionality · Naive Bayes · Bayes' rule · prior · likelihood · posterior · conditional independence · decision tree · node · leaf · split · Gini impurity · entropy · `max_depth` · `min_samples_leaf` · random forest · bagging · bootstrap sample · feature importance · gradient boosting · residual fitting · stump · learning rate · XGBoost · LightGBM · CatBoost · support vector machine · margin · support vector · kernel · RBF kernel · class weight · bias · variance · underfitting · overfitting · learning curve · validation curve · hyperparameter · grid search · random search · Bayesian optimization · Optuna · AutoML · stacking · early stopping

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 38, Unsupervised Learning**, segments the same accounts without a target, using distance and PCA.
- **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, turns churn and lead scores into decisions with cost-based thresholds, handles imbalance with class weights and resampling,

checks calibration, explains predictions with SHAP and partial dependence, and checks fairness.

- **Chapter 40, Time Series & Forecasting**, uses gradient boosting with lag features for demand forecasting.
- **Chapter 41, NLP Foundations**, uses Naive Bayes and logistic regression on text.
- **Chapter 43, A First Look at Deep Learning**, takes logistic regression's weighted sum and sigmoid, and stacks them into a neural network.
- **Chapter 22** (statistical inference) and **Chapter 31** (causal inference) cover confidence intervals for coefficients and estimating causes.
- **Interview preparation**: the Machine Learning Question Bank (Chapter 74) covers every algorithm here, bias and variance, regularization, and tuning with graded answers. "Explain gradient boosting to a non-technical person" and "random forest vs gradient boosting" are among the most common questions in data science interviews.

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. probability = $1 \div (1 + e^{-0.8}) = 1 \div (1 + 0.4493) = \mathbf{0.690}$. A probability of 0.5 needs $1 + e^{-z} = 2$, so $e^{-z} = 1$ and $\mathbf{z = 0}$. Any account whose weighted sum is exactly zero sits on the fence.

2. Odds = probability $\div$ (1 - probability) = $0.10 \div 0.90 = 0.1111$. One more complaint multiplies the odds by 1.45: $0.1111 \times 1.45 = \mathbf{0.1611}$. Back to a probability: odds $\div$ (1 + odds) = $0.1611 \div 1.1611 = \mathbf{0.139}$, or 13.9%. The common mistake is to multiply the probability itself ($10\% \times 1.45 = 14.5\%$); odds ratios multiply odds, not probabilities. The difference is small at low probabilities and large at high ones.

3. $p = 50 \div 200 = 0.25$. Gini = $2 \times 0.25 \times 0.75 = \mathbf{0.375}$. Entropy = $-(0.25 \times \log_2 0.25 + 0.75 \times \log_2 0.75) = -(0.25 \times (-2) + 0.75 \times (-0.4150)) = 0.5 + 0.3113 = \mathbf{0.811 \text{ bits}}$.

4. (a) A **shallow decision tree** (depth 2 or 3): with 300 rows, complex models overfit, and a tree's rules can be printed and followed by hand. (b) **Gradient boosting**: plenty of rows, mixed features, and accuracy as the goal, with logistic regression as the baseline to beat. (c) **Logistic regression or Naive Bayes on word features** (TF-IDF, Chapter 41), or a linear SVM: text gives thousands of sparse features, where linear models are fast and strong. (d) **Linear regression** (regularized if there are many features) on a log target: finance needs each driver's effect stated, which coefficients provide.

5.

```

REG_NUMS_FULL = REG_NUMS
REG_NUMS = [c for c in REG_NUMS_FULL if c != "log_revenue_2024"]
names_wo = (
    reg_pipeline(LinearRegression())
    .fit(reg_train[CATS + REG_NUMS], reg_train["log_revenue_2025"])
    .named_steps["prepare"]
    .get_feature_names_out()
)
for label, model in [
    ("ridge", RidgeCV(alphas=np.logspace(-3, 3, 25))),
    ("lasso", LassoCV(cv=5, random_state=37)),
]:
    fitted = reg_report(label + " without revenue", reg_pipeline(model))
    c = pd.Series(fitted.named_steps["model"].coef_, index=names_wo)
    print(
        f"    log_units_2024 {c['num__log_units_2024']:.3f}"
        f"    log_orders_2024 {c['num__log_orders_2024']:.3f}"
    )
REG_NUMS = REG_NUMS_FULL

```

```

ridge without revenue R² 0.9473    MAE ₹99,755
log_units_2024 0.987    log_orders_2024 0.074
lasso without revenue R² 0.9473    MAE ₹99,626
log_units_2024 0.991    log_orders_2024 0.074

```

Without revenue, the model leans on units instead: its coefficient jumps from nearly 0 to about 0.99, and R^2 drops only from 0.959 to 0.947, because units carry almost the same information. That's multicollinearity in action. The coefficient of a feature depends on which of its "twins" are also in the model, so a coefficient near zero doesn't mean a feature is useless; it may mean another feature is doing its job.

6.

```

clf_report("k-NN, k=75, uniform", clf_pipeline(KNeighborsClassifier(n_neighbors=75)))
clf_report(
    "k-NN, k=75, distance-weighted",
    clf_pipeline(KNeighborsClassifier(n_neighbors=75, weights="distance")),
)

```

```

k-NN, k=75, uniform          AUC 0.775    log loss 0.3057
k-NN, k=75, distance-weighted AUC 0.778    log loss 0.3050

```

Distance weighting changes the ranking only slightly here. With 75 neighbors in 25 dimensions, most neighbors are at similar distances (the curse of dimensionality again), so weighting them by distance makes little difference.

7.

```

y_all = train_acc["churned_2025"]
for label, rule in [
    ("categories_bought >= 3", train_acc["categories_bought"] >= 3),
    ("days_since_last_order > 90", train_acc["days_since_last_order"] > 90),
]:
    after = sum(len(g) / len(y_all) * gini(g) for g in (y_all[rule], y_all[~rule]))
    print(
        f"{label:<28} weighted Gini after split {after:.4f}    (before {gini(y_all):.4f})"
    )

```

```

categories_bought >= 3      weighted Gini after split 0.1710    (before 0.1746)
days_since_last_order > 90 weighted Gini after split 0.1532    (before 0.1746)

```

Splitting on three or more categories reduces Gini, but much less than the recency split, so a tree would still start with recency. The product-breadth signal is real (odds ratio 0.57) but spread across all accounts, whereas recency separates a small, high-risk group sharply. A Gini split rewards sharp separation.

8.

```

for n in [10, 50, 300, 1000]:
    clf_report(
        f"random forest, {n} trees",
        clf_pipeline(
            RandomForestClassifier(n_estimators=n, min_samples_leaf=5, random_state=37),
            scale=False,
        ),
    )

```

```

random forest, 10 trees      AUC 0.737    log loss 0.5069
random forest, 50 trees      AUC 0.774    log loss 0.2719
random forest, 300 trees     AUC 0.785    log loss 0.2675
random forest, 1000 trees    AUC 0.790    log loss 0.2659

```

Ten trees are clearly too few (AUC 0.737, with poor probabilities). The score rises with more trees and settles: going from 300 to 1,000 trees adds only 0.005. Adding trees doesn't cause overfitting in a random forest, because each tree is averaged in, not stacked on top; small ups and downs between sizes are noise from which random samples the trees drew. The only cost of more trees is time and memory.

9.

```

pred = np.full(len(y_toy), y_toy.mean())
for r in range(1, 21):
    residual = y_toy - pred
    stump = DecisionTreeRegressor(max_depth=1, random_state=37).fit(x_toy, residual)
    pred = pred + 0.5 * stump.predict(x_toy)
    if r % 5 == 0:
        print(f"round {r:>2}: squared error {np.mean((y_toy - pred) ** 2):.5f}")
print("predictions:", pred.round(3))

```

```
round 5: squared error 0.05239
round 10: squared error 0.02445
round 15: squared error 0.01143
round 20: squared error 0.00535
predictions: [0.032 0.032 0.807 0.022 0.022 0.032 0.022 0.032]
```

The error keeps falling toward zero: with enough rounds, boosting fits the eight training rows almost perfectly, including the one churner. But eight rows can't support a model that precise; it has memorized them. That's why real boosting needs a small learning rate, limits on tree size and leaf size, and a validation set or early stopping to decide when to stop adding trees. Training error going to zero is the symptom of variance from section 37.10.

10.

```
from sklearn.model_selection import validation_curve

depths = [2, 3, 4, 6, 8]
tr_scores, cv_scores = validation_curve(
    clf_pipeline(
        HistGradientBoostingClassifier(
            learning_rate=0.05, min_samples_leaf=40, max_iter=300, random_state=37
        ),
        scale=False,
    ),
    X_rest,
    y_rest,
    param_name="model__max_depth",
    param_range=depths,
    cv=folds,
    scoring="roc_auc",
)
for d, a, b in zip(depths, tr_scores.mean(axis=1), cv_scores.mean(axis=1)):
    print(f"max_depth {d}: train AUC {a:.3f}   CV AUC {b:.3f}")
```

```
max_depth 2: train AUC 0.901   CV AUC 0.836
max_depth 3: train AUC 0.945   CV AUC 0.836
max_depth 4: train AUC 0.977   CV AUC 0.829
max_depth 6: train AUC 0.999   CV AUC 0.820
max_depth 8: train AUC 1.000   CV AUC 0.817
```

The cross-validated score is best with shallow trees and declines as depth grows, while the training score keeps rising: deeper trees fit the training folds better and generalize worse. With about 390 churners in the 4,000 accounts, shallow trees (depth 2 or 3) are the right size, which matches what random search found.

11.

```

from sklearn.ensemble import StackingClassifier

stack = StackingClassifier(
    estimators=[
        ("logistic", clf_pipeline(LogisticRegression(max_iter=2000))),
        ("boosting", search.best_estimator_),
    ],
    final_estimator=LogisticRegression(max_iter=2000),
    cv=5,
)
s = cross_val_score(stack, X_rest, y_rest, cv=folds, scoring="roc_auc")
print(f"stacking mean AUC {s.mean():.3f} sd {s.std():.3f}")
print(f"(tuned boosting alone {search.best_score_:.3f}; logistic alone 0.807)")

```

```

stacking mean AUC 0.840 sd 0.016
(tuned boosting alone 0.841; logistic alone 0.807)

```

The stack scores about the same as tuned boosting alone. Stacking helps when the models make different kinds of mistakes and each is strong; here, boosting already captures what logistic regression knows, so the combination adds little, at double the training time and complexity. That's the usual outcome in business problems, and why stacking is more common in competitions than in production.

12.

```

boost_eng = Pipeline(
    [
        (
            "prepare",
            ColumnTransformer(
                [
                    ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
                    (
                        "num",
                        SimpleImputer(strategy="median", add_indicator=True),
                        NUMS + ENG,
                    ),
                ],
            ),
        ),
        (
            "model",
            HistGradientBoostingClassifier(
                **{k.replace("model__", ""): v for k, v in search.best_params_.items()},
                random_state=37,
            ),
        ),
    ]
)
s = cross_val_score(
    boost_eng, engineered[CATS + NUMS + ENG], y_rest, cv=folds, scoring="roc_auc"
)
print(
    f"tuned boosting + engineered features mean AUC {s.mean():.3f} sd {s.std():.3f}"
)

```

```

tuned boosting + engineered features mean AUC 0.846 sd 0.013

```

Boosting gains only a little from the engineered features (0.846 against 0.841), far less than logistic regression's jump from 0.807 to 0.849. Trees can already represent thresholds ("days > 90") and interactions (late payment within small retail) by splitting; the engineered features mostly save them a few splits. Logistic regression couldn't represent them at all, so for it they were new information. Feature engineering matters most for the models that can't find structure on their own.

13.

```
import statsmodels.api as sm

design = pd.get_dummies(reg_train[CATS], drop_first=True, dtype=float)
design = pd.concat(
    [design, reg_train[REG_NUMS].fillna(reg_train[REG_NUMS].median())], axis=1
)
ols_sm = sm.OLS(reg_train["log_revenue_2025"], sm.add_constant(design)).fit()
low, high = ols_sm.conf_int().loc["rep_id_9"]
print(
    f"inside sales desk (rep_id_9): coefficient {ols_sm.params['rep_id_9']:.4f}    "
    f"95% CI [{low:.4f}, {high:.4f}]    p-value {ols_sm.pvalues['rep_id_9']:.3f}"
)
```

```
inside sales desk (rep_id_9): coefficient 0.0311    95% CI [0.0092, 0.0531]    p-value 0.006
```

The interval, 0.009 to 0.053, **excludes zero** ($p = 0.006$), even though the generator gave the inside sales desk no effect on growth. So where does it come from? The generator did make inside-desk accounts **more likely to churn**, and this regression only includes accounts that stayed. Inside-desk accounts that survived despite that extra risk tend to be healthier on other churn signals, such as the late-payment and complaint thresholds, which the model's straight-line terms don't fully capture. That's **selection bias**: analyzing only the survivors creates a relationship that doesn't exist among all accounts (Chapter 31). The lesson is broader than this dataset. A statistically significant coefficient can still be an artifact of how the rows were chosen, so before anyone reassigns accounts to the inside sales desk, ask how the data was filtered. (statsmodels uses unscaled features, so its coefficients are in original units, unlike the scaled ones in section 37.1; for a one-hot column like this one, the meaning is the same.)

14. First, the scores come from **different evaluations**: 0.841 is the best of 25 tuned settings on cross-validation, which is slightly optimistic because the winner was selected for scoring well, while 0.807 is logistic regression's untuned CV score. Compare like with like, ideally on the test set (0.839 against 0.797). Second, a difference in **AUC points isn't a percentage improvement** in anything a business cares about: 0.034 AUC doesn't mean 4% more churners saved. Translate it into decisions, such as "how many of the 100 highest-risk accounts actually churn", which is what Chapter 39 does.

15. Impurity-based importance adds up how much each feature reduces Gini across all splits. A continuous feature with thousands of distinct values, like `revenue_2024`, offers many possible thresholds, so trees use it for many small splits that fit noise, and it accumulates importance. `complaints_2024` has only a few values (0 to 6), and its effect is concentrated in an interaction (new accounts with two or more complaints) that occurs in few rows, so it's used rarely. Permutation importance on validation data, or SHAP values (Chapter 39), measure how much predictions actually depend on each feature and correct most of this bias.

16. (1) Drift (Chapter 36): customers' behavior changed, for example a new competitor makes even long-standing accounts leave, so patterns learned from 2024 don't hold. **(2) Variance:** last year's evaluation was on too few accounts and was lucky; the model was never as good as it looked. **(3) A new kind of bias:** the business changed (a new product line, a pricing change) in a way the features don't capture, so the model is now too simple for the new situation. A fourth possibility is a **pipeline problem:** a feature changed meaning or started arriving empty. Check that first; it's the most common and the easiest to fix.